Mini Search Engine – File Indexing and Search using Trie, Inverted Index and Priority Queue

Mini Search Engine

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Ch Karthikeya – 2520080022
S Abhinav – 2520080043

Team Number: [3]

Section: [12]

Under the guidance of

K.Chandusha

Assistant Professor, Department of CSIT

Hyderabad-500055, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "Mini Search Engine – File Indexing and Search using Trie, Inverted Index and Priority Queue" is a record of Bonafide work of Ch Karthikeya – 2520080022 and S Abhinav – 2520080043 submitted in partial fulfillment for the award of B. Tech in Artificial Intelligence & Data Science to the K L University. The results embodied in this report have been developed as part of the Data Structures and Algorithms -3 project work.

Ch Karthikeya – 2520080022

S Abhinav – 2520080043

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "Mini Search Engine – File Indexing and Search using Trie, Inverted Index and Priority Queue" is a bonafide work done and submitted by Ch Karthikeya – 2520080022 and S Abhinav – 2520080043 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year 2026-2027.

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

We sincerely express our gratitude to everyone who supported us during the development and completion of this Data Structures and Algorithms project.

We are especially thankful to our project guide, [Guide Name], [Designation, Department], for providing valuable guidance, technical suggestions and encouragement throughout the project.

We also thank the Department of Artificial Intelligence & Data Science and the faculty members of K L (Deemed to be) University for providing the academic environment and facilities required to complete this work.

Finally, we thank our parents, friends and classmates for their cooperation and support during the design, implementation, testing and documentation of the Mini Search Engine.

Name – Roll.No.

Ch Karthikeya – 2520080022

S Abhinav – 2520080043

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	9
5.	Hardware and Software requirements	10
6.	Implementation	11
7.	Results & Discussion	12
8.	Conclusion & Future Scope	13
9.	References	14

1.ABSTRACT

This project presents a Mini Search Engine implemented in Java to demonstrate how core Data Structures and Algorithms can be combined to build an efficient document-search system. The application indexes text documents and maintains an in-memory inverted index using a HashMap that maps keywords to the documents in which they occur. A Trie is used to store vocabulary and provide prefix-based autocomplete suggestions. During a search, the SearchEngine processes query tokens, retrieves matching documents from the inverted index and calculates relevance using keyword frequency and filename matching. The resulting documents are ranked using Java PriorityQueue configured as a Max-Heap, with document name used as a deterministic tie-breaker. The application also demonstrates Stack-based LIFO search history and a bounded Queue-based FIFO recent-search window. It includes a console application and a lightweight Java HTTP server with a browser-based frontend for search, autocomplete, file browsing, indexing and performance telemetry. An automated test suite validates ranking, Trie autocomplete, Stack history, Queue eviction and end-to-end search. The supplied test execution completed successfully with 48 tests passed and 0 failures. The project demonstrates how appropriate data structures can organize indexing, retrieval, ranking and user interaction into a modular search system.

Team Name: Mini Search Engine Team

Team Logo (if any): Not applicable

Team Members:

1. Ch Karthikeya – 2520080022
2. S Abhinav – 2520080043

2. INTRODUCTION

The Mini Search Engine is a Java-based document indexing and retrieval application developed as a Data Structures and Algorithms project. Its purpose is to demonstrate how different data structures can be combined to solve a practical search problem instead of repeatedly scanning every document from the beginning.

The system reads text and code files from a selected directory and creates an in-memory index. The Indexer maintains a HashMap-based inverted index in which each normalized keyword points to the documents containing that keyword. A Document object stores file metadata, content, word count and keyword frequencies.

A Trie is maintained alongside the inverted index to support exact vocabulary lookup, prefix detection and autocomplete. When the user enters a prefix, the Trie locates the corresponding prefix node and collects candidate words, which are then ordered using metadata such as file priority, document count and frequency.

Search results are ranked using a Java PriorityQueue configured with a custom comparator so that higher-frequency results receive higher priority. The project also uses a Stack for LIFO search history and a fixed-capacity Queue for recent searches, allowing several core DSA concepts to work together in one application.

The system is available through a console application and a lightweight HTTP web interface. The web version exposes search, autocomplete, statistics, document inspection, folder browsing and indexing operations through Java HTTP handlers and a browser frontend. The following sections describe the related work, problem definition, requirements, implementation, results and future improvements.

3. LITERATURE SURVEY

1. Apache Lucene: Apache Lucene is a mature Java search library built around high-performance indexing and ranked retrieval. Its architecture uses inverted indexes and supports query processing, ranking and suggestions. It is relevant because it demonstrates the same broad search-engine principle used here: build an index once and use it for efficient retrieval.

2. Elasticsearch Full-Text Search: Elasticsearch creates an inverted index after text analysis, mapping terms to the documents that contain them. It also calculates relevance scores for returned documents. This provides a real-world reference for the inverted-index and relevance-ranking concepts demonstrated by the Mini Search Engine.

3. MiniSearch: MiniSearch is a small in-memory full-text search engine for JavaScript. It supports exact and prefix search, fuzzy matching, ranking and autosuggestions. It is similar in spirit to this project because both emphasize local in-memory search and interactive query completion.

4. From-scratch Mini Search Engine: An educational search engine implementation combines inverted indexing, Trie autocomplete and heap-based Top-K retrieval. It shows how multiple DSA components can cooperate to avoid repeated document scanning and to rank relevant results.

5. Java Mini Google Search Engine: This educational Java project combines inverted indexing, Trie-based suggestions, search history, ranking and performance statistics. Its modular organization is comparable to the present project and reinforces the value of combining indexing, autocomplete and user-interaction structures.

6. MiniSearchEngine using DSA primitives: This project implements an inverted index, Trie, heap ranking and advanced query features from core data structures. It demonstrates how search-engine components can be separated into indexing, retrieval and ranking modules.

7. Java PriorityQueue: The Java Collections Framework provides PriorityQueue as a heap-based priority queue with logarithmic insertion and removal operations and constant-time access to the head. The present project uses this structure as a Max-Heap for relevance ranking.

8. Java Queue Implementations: Java's LinkedList can provide FIFO queue behavior, while PriorityQueue provides priority ordering. The present project uses LinkedList through the Queue interface for a bounded recent-search window, illustrating FIFO eviction

4. Problem Statement and Objectives

Problem Statement: Build a fast document search system that retrieves and ranks matching files using suitable data structures instead of repeated linear scanning.

The system should support keyword search, ranked results, autocomplete and recent-search management.

Problem Description: Scanning every file for every query becomes inefficient as a collection grows. The project preprocesses documents into an in-memory inverted index and combines it with Trie autocomplete and heap-based ranking.

The design also demonstrates Stack-based history and a bounded FIFO Queue.

Objectives:

1. Build a HashMap-based inverted index for keyword-to-document retrieval.

2. Implement a Trie for vocabulary indexing and prefix autocomplete.

3. Rank results with PriorityQueue configured as a Max-Heap.

4. Demonstrate Stack LIFO history, Queue FIFO recent searches and automated testing.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- • Dual-core processor or better
- • Minimum 4 GB RAM
- • At least 1 GB free storage

• Keyboard, mouse and standard display

- • Java Development Kit (JDK 11 or higher; README reports testing with Java 26)
- • IntelliJ IDEA or another Java-compatible IDE
- • Windows/Linux/macOS operating system
- • Modern web browser for the web interface

• Git/GitHub for source-code version control (optional)

• No external database is required; indexing is performed in memory

6. IMPLEMENTATION

The implementation is organized into four DSA-focused modules:

1. Indexing & Trie – Indexer.java builds Document objects, word frequencies, the inverted index and Trie vocabulary. Trie.java provides prefix autocomplete. 2. Search & Max-Heap – SearchEngine.java retrieves matches and computes scores; Ranker.java uses PriorityQueue to return highest-score results first. 3. Stack & Queue – History.java stores LIFO search history; RecentSearchQueue.java stores the latest five queries using FIFO eviction. 4. Interface – UI.java provides console interaction; WebServer.java and web/ provide browser-based search, indexing and telemetry.

Complexity: HashMap lookup is O(1) average; Trie prefix traversal is O(L); PriorityQueue insertion/removal is O(log N); Stack push/peek and Queue offer/poll are O(1).

7. RESULTS & DISCUSSION

Testing and execution were completed successfully. The automated suite covered ranking, tie-breaking, Stack history, Queue eviction, Trie autocomplete and end-to-end search: 48 tests passed and 0 failed.

1. Indexing: The demo corpus contains 3 text documents and the server reports 35 unique indexed words.

2. Search: Query “java” returns book1.txt as the top result with score 3.

3. Autocomplete: Prefix “pro” returned “programming”, “provide” and “provides”.

4. Ranking: Results are ordered by descending score, with filename used as the deterministic tie-breaker.

5. History/Queue: Stack gives latest-first history; the five-item Queue removes the oldest entry when full.

6. Telemetry: The web endpoint reports total time, inverted-index lookup time, heap-ranking time and match count.

8. CONCLUSION & FUTURE SCOPE

The Mini Search Engine successfully integrates an inverted index, Trie, PriorityQueue Max-Heap, Stack and Queue into a practical search application. It supports document indexing, keyword retrieval, relevance ranking, autocomplete, history and recent searches through both console and web interfaces. The supplied automated suite passed all 48 tests, confirming the major components and their end-to-end integration.

Future Scope:

1. Add TF-IDF/BM25 ranking for more advanced relevance scoring.
2. Add persistent indexes, phrase search, Boolean queries and fuzzy matching.
3. Extend indexing to PDF, DOCX and other structured formats.
4. Improve autocomplete and add larger benchmark datasets with latency and retrieval metrics.
5. Support concurrent indexing and thread-safe search for larger collections.

9. REFERENCES

1. Apache Lucene, “Apache Lucene Core – High-performance search library.” https://lucene.apache.org/core/

2. Elastic, “Full-text search – Inverted index and relevance scoring.” https://www.elastic.co/guide/en/elasticsearch/reference/8.19/full-text-search.html

3. Oracle, “PriorityQueue (Java SE API).” https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/util/PriorityQueue.html

4. Luca Ongaro, “MiniSearch – Tiny and powerful JavaScript full-text search engine.” https://github.com/lucaong/minisearch

5. Shivansh Sharma, “MiniGoogleSearchEngine – Java mini search engine.” https://github.com/shivanshsharmadev21/MiniGoogleSearchEngine